

## 1. 보고 목적

본 문서는 최종 프로젝트에서 담당한 프론트엔드 영역의 주요 구현 내용을 정리한 문서이다. 프론트엔드는 사용자와 직접 상호작용하는 화면을 구성하고, 로그인 상태 관리, 채팅 세션 처리, 문서 업로드 및 요약 결과 표시, 웹소켓 기반 실시간 통신 기능을 담당한다.

전체 프론트엔드 코드는 페이지, 상태관리, API 통신, 공통 UI 컴포넌트 등으로 구성되어 있으나, 본 문서에서는 그중 핵심 기능 흐름을 담당하는 주요 소스코드를 중심으로 설명하고자 한다.

## 2. 작업 범위

- React + Vite + TypeScript 기반 프론트엔드 구조 구성
- 사용자 인증 흐름 및 권한별 화면 분기
- 채팅 화면 UI 및 채팅 세션 관리
- 문서 업로드 및 문서 요약 결과 표시 기능
- WebSocket 기반 실시간 채팅 연동
- 전역 상태관리 및 공통 UI 동작 처리
- 문서 검색, 채팅 검색, 사이드바 제어 등 사용자 편의 기능 구현

## 3. 프론트엔드 구조 요약

프론트엔드는 기능별 책임 분리를 고려하여 다음과 같은 구조로 구성하였다.

- **main.tsx**  
프론트엔드 애플리케이션의 진입점으로, React 애플리케이션을 브라우저에 렌더링하고 라우터를 연결하는 역할을 수행한다.
- **App.tsx**  
전체 라우팅을 관리하며, 로그인 여부와 사용자 권한에 따라 일반 사용자 화면과 관리자 화면으로 분기하는 역할을 수행한다.
- **pages**  
실제 사용자에게 노출되는 화면 단위 컴포넌트로 구성되며, 채팅 페이지, 문서 페이지, 로그인 페이지 등이 포함된다.
- **layouts**  
화면의 공통 구조를 담당하는 레이아웃 계층으로, 사이드바와 메인 콘텐츠 영역을 구성한다.

- **stores**

Zustand 기반 전역 상태관리 계층으로, 인증 상태, 채팅 상태, UI 상태, 문서 상태 등을 관리한다.

- **lib**

백엔드 API 통신, 토큰 처리, 웹소켓 연결 등 외부 연동 기능을 담당하는 모듈로 구성하였다.

## 4. 주요 소스코드 설명

### 4.1 main.tsx

```
8   ReactDOM.createRoot(document.getElementById("root")!).render(  
9     <React.StrictMode>  
10       <BrowserRouter>  
11         <App />  
12       </BrowserRouter>  
13     </React.StrictMode>
```

#### ▲ 애플리케이션 진입 및 라우팅 설정

main.tsx는 프론트엔드 애플리케이션의 실행 진입점을 담당하는 파일이다.

해당 파일에서는 ReactDOM을 이용하여 애플리케이션을 루트 DOM에 렌더링하고, BrowserRouter를 적용하여 페이지 라우팅이 가능하도록 구성하였다. 또한 전역 스타일과 아이콘 스타일을 함께 로드하여 전체 화면의 기본 UI 환경을 초기화하도록 하였다.

즉, 본 파일은 프론트엔드 애플리케이션이 동작하기 위한 기본 실행 구조를 담당하며, 다른 기능성 코드와 분리하여 프로젝트 구조를 명확하게 유지할 수 있도록 하였다.

## 4.2 App.tsx

```
36 function ProtectedChatRoute() {
37   const { isSignedIn, isHydrating, user } = useAuthStore();
38
39   if (isHydrating) return <AuthLoadingScreen />;
40   if (!isSignedIn) return <Navigate to="/signin" replace />;
41   if (user?.userType === "ADMIN") return <Navigate to="/admin" replace />;
42
43   return <ChatLayout />;
44 }
45
46 function ProtectedAdminRoute() {
47   const { isSignedIn, isHydrating, user } = useAuthStore();
48
49   if (isHydrating) return <AuthLoadingScreen />;
50   if (!isSignedIn) return <Navigate to="/signin" replace />;
51   if (user?.userType !== "ADMIN") return <Navigate to="/chat" replace />;
52
53   return <AdminLayout />;
```

### ▲ 보호 라우트 및 권한 분기

해당 코드는 로그인 여부와 사용자 권한에 따라 접근 가능한 화면을 분기하는 로직이다. 비로그인 사용자는 로그인 페이지로 이동하도록 처리하고, 관리자와 일반 사용자의 접근 경로를 구분하여 권한에 맞는 화면만 진입할 수 있도록 구현하였다.

```

66 export default function App() {
67   const hydrateUser = useAuthStore((s) => s.hydrateUser);
68
69   useEffect(() => {
70     hydrateUser();
71   }, [hydrateUser]);
72
73   return (
74     <div className="min-h-screen bg-background text-foreground">
75       <ToastViewport />
76
77       <Routes>
78         <Route path="/" element={<RootRedirect />} />
79         <Route path="/signin" element={<SignInPage />} />
80         <Route path="/signup" element={<SignUpPage />} />
81
82         <Route path="/auth/kakao/callback" element={<KakaoCallbackPage />} />
83
84         <Route path="/chat" element={<ProtectedChatRoute />}>
85           <Route index element={<ChatRouterPage />} />
86           <Route path=":sessionId" element={<ChatRouterPage />} />
87         </Route>
88
89         <Route path="/admin" element={<ProtectedAdminRoute />}>
90           <Route index element={<AdminDashboardPage />} />
91           <Route path="users" element={<AdminUsersPage />} />
92           <Route path="documents" element={<AdminDocumentsPage />} />
93           <Route path="chats" element={<AdminChatsPage />} />
94           <Route path="systems" element={<AdminSystemsPage />} />
95         </Route>
96       </Routes>

```

#### ▲ 전체 페이지 라우팅 구성

해당 코드는 애플리케이션의 전체 라우팅 구조를 정의한 부분이다. 회원가입, 로그인, 채팅, 관리자 페이지 등 주요 경로를 구성하고, 각 경로가 보호 라우트와 연결되도록 설정하였다.

App.tsx 는 전체 프론트엔드의 라우팅과 접근 제어를 담당하는 핵심 파일이다. 애플리케이션 실행 시 hydrateUser()를 호출하여 저장된 로그인 상태를 복구하고, 이후 사용자 인증 여부와 권한에 따라 /signin, /chat, /admin 경로로 분기되도록 구현하였다. 이를 통해 라우팅 로직과 권한 제어 로직을 한 곳에서 일관되게 관리할 수 있도록 구성하였다.

### 4.3 authStore.ts

```
35   signin: async ({ email, password }) => {
36     useChatStore.getState().reset();
37     useUiStore.getState().reset();
38
39     const tokens = await loginApi({ email, password });
40
41     try {
42       setTokens({
43         accessToken: tokens.accessToken,
44         refreshToken: tokens.refreshToken,
45       });
46       // userInfo 조회는 apiFetch를 통해 호출되므로
47       // access token이 만료되어도 refresh 후 자동 재시도된다
48       const UserInfo = await getUserInfo();
49
50       const user: User = {
51         name: UserInfo.name,
52         email: UserInfo.email,
53         userType: UserInfo.user_type,
54       };
55
56       set({
57         isSignedIn: true,
58         isHydrating: false,
59         accessToken: getAccessToken(),
60         refreshToken: getRefreshToken(),
61         user,
62       });
63
64       return user;
65     } catch (error) {
66       clearTokens();
67       set({
68         isSignedIn: false,
69         isHydrating: false,
70         user: null,
71         accessToken: null,
72         refreshToken: null,
73       });
74       throw error;
75     }
76   }
77 }
```

#### ▲로그인 처리

해당 코드는 사용자가 로그인에 성공했을 때 액세스 토큰과 리프레시 토큰을 저장하고, 사용자 정보를 조회하여 전역 상태에 반영하는 로직이다. 또한 로그인 이전의 채팅 상태와 UI 상태를 초기화하여 새로운 사용자 세션이 일관된 상태에서 시작될 수 있도록 구현하였다.

```

118 hydrateUser: async () => {
119   const accessToken = getAccessToken();
120   const refreshToken = getRefreshToken();
121
122   set({ isHydrating: true });
123
124   if (!accessToken) {
125     useChatStore.getState().reset();
126     useUiStore.getState().reset();
127
128     set({
129       isSignedIn: false,
130       isHydrating: false,
131       user: null,
132       accessToken: null,
133       refreshToken: null,
134     });
135     return;
136   }
137
138   try {
139     // 새로고침 후에도 /userInfo 호출 시 refresh 흐름이 자동 적용된다
140     const me = await getUserInfo();
141
142     set({
143       isSignedIn: true,
144       isHydrating: false,
145       accessToken: getAccessToken(),
146       refreshToken: getRefreshToken() ?? refreshToken,
147       user: {
148         name: me.name,
149         email: me.email,
150         userType: me.user_type,
151       },
152     });
153   } catch {
154     useChatStore.getState().reset();
155     useUiStore.getState().reset();
156     clearTokens();
157     set({
158       isSignedIn: false,
159       isHydrating: false,
160       user: null,
161       accessToken: null,
162       refreshToken: null,
163     });
164   }

```

#### ▲자동 로그인 복구

해당 코드는 애플리케이션 재실행 시 저장된 토큰 정보를 기반으로 로그인 상태를 복구하는 로직이다. 액세스 토큰이 존재하지 않는 경우에는 인증 상태를 초기화하고, 토큰이 유효한 경우에는 사용자 정보를 다시 조회하여 로그인 상태를 유지하도록 구현하였다.

authStore.ts 는 사용자 인증 상태를 전역으로 관리하기 위한 Zustand 스토어이다. 주요 기능으로는 로그인 처리, 소셜 로그인 완료 처리, 로그인 상태 복구, 로그아웃, 회원탈퇴, 사용자 정보 수정 등이 있다. 이와 같이 인증 관련 로직을 중앙에서 통합 관리함으로써 각 페이지에서 인증 처리 로직이 중복되지 않도록 구성하였다.

#### 4.4 apiClient.ts

```
13  async function refreshAccessToken(): Promise<string | null> {
14    const refreshToken = getRefreshToken();
15
16    if (!refreshToken) {
17      clearTokens();
18      return null;
19    }
20
21    if (!refreshPromise) {
22      refreshPromise = refresh(refreshToken)
23        .then((tokens) => {
24          setTokens({
25            accessToken: tokens.accessToken,
26            refreshToken: tokens.refreshToken,
27          });
28          return tokens.accessToken;
29        })
30        .catch(() => {
31          // refresh 자체가 실패하면 세션을 정리하고 다시 로그인하게 만든다
32          clearTokens();
33          return null;
34        })
35        .finally(() => {
36          refreshPromise = null;
37        });
38    }
39
40    return refreshPromise;
```

##### ▲토큰 재발급 로직

해당 코드는 리프레시 토큰을 이용하여 새로운 액세스 토큰을 발급받는 로직이다. 리프레시 토큰이 없거나 재발급에 실패한 경우에는 인증 정보를 초기화하여 다시 로그인할 수 있도록 처리하였다.

```

43 export async function apiFetch(path: string, init: RequestInit = {}) {
44   const accessToken = getAccessToken();
45
46   const response = await fetch(`${API_BASE_URL}${path}`, {
47     ...init,
48     headers: {
49       ...(init.body instanceof FormData ? {} : { "Content-Type": "application/json" }),
50       ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
51       ...(init.headers ?? {}),
52     },
53   });
54
55   // 401이 아니면 그대로 반환한다
56   if (response.status !== 401) {
57     return response;
58   }
59
60   const newAccessToken = await refreshAccessToken();
61
62   if (!newAccessToken) {
63     throw new Error("세션이 만료되었습니다. 다시 로그인해주세요.");
64   }
65
66   // 새 access token으로 원래 요청을 한 번만 재시도한다.
67   return fetch(`${API_BASE_URL}${path}`, {
68     ...init,
69     headers: {
70       ...(init.body instanceof FormData ? {} : { "Content-Type": "application/json" }),
71       Authorization: `Bearer ${newAccessToken}`,
72       ...(init.headers ?? {}),
73     },
74   });

```

#### ▲API 재요청 로직

해당 코드는 API 요청 수행 중 401 Unauthorized 응답이 발생한 경우, 새로 발급받은 액세스 토큰으로 기존 요청을 다시 수행하는 로직이다. 이를 통해 인증이 필요한 API 통신이 중단되지 않도록 구성하였다.

apiClient.ts는 백엔드 API 통신의 공통 처리를 담당하는 모듈이다. 모든 API 요청은 apiFetch()를 통해 수행되며, 요청 시 액세스 토큰을 자동으로 헤더에 포함하도록 구성하였다. 또한 동시에 여러 요청이 발생하더라도 refresh 요청이 중복 실행되지 않도록 Promise를 공유하는 방식으로 처리하여, 인증이 필요한 API 통신을 안정적으로 수행할 수 있도록 하였다.



## 4.5 chatStore.ts

```
109     loadSessions: async () => {
110       set({ isLoadingSessions: true });
111
112       try {
113         const data = await getSessions();
114
115         set({
116           sessions: sortSessionsByUpdatedAt(
117             data.map((item) => ({
118               id: item.session_id,
119               title: item.title,
120               updatedAt: item.updated_at,
121             })))
122         ),
123       };
124     } catch (error) {
125       const message =
126         error instanceof Error ? error.message : "세션 목록을 불러오지 못했습니다.";
127
128       showToast({
129         title: "채팅 세션 조회 실패",
130         description: message,
131         tone: "error",
132       });
133     } finally {
134       set({ isLoadingSessions: false });
135     }
136   },
```

### ▲ 채팅 세션 목록 조회

해당 코드는 채팅 세션 목록을 서버에서 조회하고, 로딩 상태와 세션 정보를 전역 상태에 반영하는 로직이다. 서버에서 받은 세션 데이터를 화면에서 사용할 수 있는 형태로 변환한 뒤 최신 순으로 정렬하여 관리하도록 구현하였다. 또한 조회 실패 시에는 토스트 알림을 통해 사용자에게 오류를 안내하도록 처리하였다.

```

283   connectSocket: async (sessionId) => {
284     const { disconnectSocket } = get();
285
286     disconnectSocket();
287
288     try {
289       const socket = connectChatSocket(sessionId);
290
291       await new Promise<void>((resolve, reject) => {
292         socket.onopen = () => {
293           set({ socket });
294           resolve();
295         };
296
297         socket.onerror = () => {
298           reject(new Error("웹소켓 연결 중 문제가 발생했습니다."));
299         };
300       });
301
302       socket.onmessage = (event) => {
303         try {
304           const data = JSON.parse(event.data);
305
306           if (data.type === "session_created") {
307             set((state) => {
308               const nextSessions = [
309                 {
310                   id: data.session_id,
311                   title: data.title,
312                   updatedAt: data.created_at ?? new Date().toISOString(),
313                 },
314                 ...state.sessions.filter((item) => item.id !== data.session_id),
315               ];
316
317               return {
318                 activeSessionId: data.session_id,
319                 sessions: sortSessionsByUpdatedAt(nextSessions),
320                 messagesBySession: {
321                   ...state.messagesBySession,
322                   [data.session_id]: state.pendingNewChatMessages,
323                 },
324               };
325             });
326           }
327         } catch {
328           // ...
329         }
330       };
331     }
332   },
333   // ...
334 }

```

#### ▲ 웹소켓 연결 및 세션 상태 반영

해당 코드는 웹소켓 연결을 생성하고, 연결 성공 및 실패 상태를 처리하는 로직이다. 또한 서버로부터 session\_created 이벤트를 수신했을 때 새로운 채팅 세션 정보를 현재 상태에 반영하도록 구현하였다.

chatStore.ts는 채팅 기능 전반의 상태를 관리하는 핵심 스토어이다. 채팅 세션 목록, 세션별 메시지 목록, 현재 선택된 세션, 입력 중인 draft, 웹소켓 연결 상태, 메시지 전송 상태 등을 통합적으로 관리한다. 이와 같이 채팅 관련 복합 로직을 별도 스토어에서 관리함으로써 화면 컴포넌트의 복잡도를 낮추고, 상태 흐름을 보다 명확하게 유지할 수 있도록 하였다.

## 4.6 chatSocket.ts

```
5  function toWsBaseUrl(httpUrl: string) {
6      if (httpUrl.startsWith("https://")) return httpUrl.replace("https://", "wss://");
7      return httpUrl.replace("http://", "ws://");
8  }
9
10 export function connectChatSocket(sessionId?: string) {
11     const token = getAccessToken();
12
13     if (!token) {
14         throw new Error("로그인이 필요합니다.");
15     }
16
17     const wsBaseUrl = toWsBaseUrl(API_BASE_URL);
18     const url = new URL(`${wsBaseUrl}/ws/chat`);
19     url.searchParams.set("token", token);
20
21     if (sessionId) {
22         url.searchParams.set("session_id", sessionId);
23     }
24
25     return new WebSocket(url.toString());
}
```

### ▲ 웹소켓 연결 생성

해당 코드는 HTTP 기반 API 주소를 WebSocket 주소로 변환하고, 인증 토큰과 세션 ID를 포함하여 채팅용 웹소켓 연결을 생성하는 로직이다. 이를 통해 서버와의 실시간 채팅 통신이 가능하도록 구현하였다.

chatSocket.ts는 채팅용 웹소켓 연결을 생성하는 모듈이다. 사용자가 특정 세션에 진입한 경우 해당 세션 ID를 기준으로 연결이 이루어지며, 인증 토큰이 없는 경우에는 연결이 차단되도록 예외 처리하였다. 이를 통해 실시간 채팅 기능에 필요한 연결 생성 책임을 별도 모듈로 분리하였다.

## 4.7 ChatPage.tsx

```
22 // assistant 답변을 한 글자씩 보여 주는 타이핑 애니메이션 컴포넌트
23 function TypingText({
24   text,
25   shouldAnimate,
26   speed = 20,
27   onDone,
28 }: {
29   text: string;
30   shouldAnimate: boolean;
31   speed?: number;
32   onDone?: () => void;
33 }) {
34   const [visibleText, setVisibleText] = useState(shouldAnimate ? "" : text);
35
36   useEffect(() => {
37     if (!shouldAnimate) {
38       setVisibleText(text);
39       return;
40     }
41
42     setVisibleText("");
43
44     let index = 0;
45     const timer = window.setInterval(() => {
46       index += 1;
47       setVisibleText(text.slice(0, index));
48
49       if (index >= text.length) {
50         window.clearInterval(timer);
51         onDone?.();
52       }
53     }, speed);
54
55     return () => window.clearInterval(timer);
56   }, [text, shouldAnimate, speed, onDone]);
57
58   const isTyping = shouldAnimate && visibleText.length < text.length;
59
60   return (
61     <p className="whitespace-pre-wrap">
62       {visibleText}
63       {isTyping ? <span className="animate-pulse"></span> : null}
64     </p>/.whitespace-pre-wrap
65   );
66 }
```

### ▲ assistant 응답 타이핑 애니메이션

해당 코드는 assistant 응답 메시지를 한 글자씩 출력하는 타이핑 애니메이션을 구현한 부분이다. 답변이 한 번에 표시되지 않고 점진적으로 나타나도록 구성하여 실제 대화와 유사한 사용자 경험을 제공하도록 하였다.

```

344 const handleFiles = (files: FileList | null) => {
345   if (isSending || isUploading) return;
346   if (draft.trim().length > 0) {
347     showToast({
348       title: "동시 첨부 불가",
349       description: "질문이 입력된 상태에서는 문서를 첨부할 수 없습니다.",
350       tone: "error",
351     });
352     return;
353   }
354   if (!files || files.length === 0) return;
355
356   const nextFiles = Array.from(files);
357
358   setPendingFiles((prev) => {
359     const merged = [...prev];
360
361     for (const file of nextFiles) {
362       const exists = merged.some(
363         (item) =>
364           item.name === file.name &&
365           item.size === file.size &&
366           item.lastModified === file.lastModified
367       );
368
369       if (!exists && merged.length >= MAX_PENDING_FILES) {
370         showToast({
371           title: "첨부 제한",
372           description: "문서는 최대 3개까지 첨부할 수 있습니다.",
373           tone: "error",
374         });
375         break;
376       }
377
378       if (!exists) {
379         merged.push(file);
380       }
381     }
382
383     return merged.slice(0, MAX_PENDING_FILES);
384   });
385 };

```

#### ▲ 문서 첨부 제한 처리

해당 코드는 문서 첨부 시 업로드 가능 여부를 확인하고, 질문이 입력된 상태에서는 첨부를 제한하는 로직이다. 또한 첨부 가능한 문서 개수를 최대 3개로 제한하여 입력 흐름이 충돌하지 않도록 구현하였다.

ChatPage.tsx는 실제 채팅 화면을 구성하는 핵심 페이지 컴포넌트이다. 사용자 메시지 입력, 메시지 목록 출력, 전송 버튼 처리, 엔터 입력 전송, 자동 스크롤, 하단 이동 버튼, 문서 첨부 및 업로드 기능 등을 포함한다. 특히 텍스트 질문 전송과 문서 업로드를 하나의 흐름 안에서 처리하되, 문서와 질문을 동시에 전송할 수 없도록 분기 로직을 적용하여 사용자 경험을 고려한 인터페이스를 제공하고자 하였다.

## 4.8 DocumentsPage.tsx

```
42 // 검색어가 있으면 파일명/요약 기준으로 실시간 필터링한다.
43 const filteredDocuments = useMemo(() => {
44   const q = documentSearchQuery.trim().toLowerCase();
45   if (!q) return documents;
46   return documents.filter((doc) => doc.name.toLowerCase().includes(q) || doc.summary.toLowerCase().includes(q));
47 }, [documents, documentSearchQuery]);
48
49 const [selected, setSelected] = useState<DocItem | null>(filteredDocuments[0] ?? null);
50
51 useEffect(() => {
52   void loadMyDocuments();
53 }, [loadMyDocuments]);
54
55 useEffect(() => {
56   // 필터 결과가 바뀌었을 때 선택 문서가 유효하지 않으면 첫 문서로 보정한다.
57   if (filteredDocuments.length === 0) {
58     setSelected(null);
59     return;
60   }
61
62   if (!selected || !filteredDocuments.some((doc) => doc.id === selected.id)) {
63     setSelected(filteredDocuments[0]);
64   }
65 }, [filteredDocuments, selected]);
```

### ▲ 문서 검색 및 선택 상태 보정

해당 코드는 검색어를 기준으로 문서 목록을 필터링하고, 선택된 문서 상태를 유효한 값으로 유지하는 로직이다. 검색 결과가 변경되더라도 사용자가 항상 현재 확인 가능한 문서를 선택할 수 있도록 보정 기능을 적용하였다.

```

83     const handleDeleteDocument = async () => {
84         if (!selected) return;
85
86         const confirmed = window.confirm(`${selected.name}" 문서를 삭제하시겠습니까?`);
87         if (!confirmed) return;
88
89         try {
90             const deletedSessionId = selected.sessionId;
91             const result = await removeDocument(selected.id);
92
93             if (result.chat_session_deleted) {
94                 const wasActiveSessionDeleted = activeSessionId === deletedSessionId;
95
96                 await loadSessions();
97
98                 const nextSessions = useChatStore.getState().sessions;
99
100                if (wasActiveSessionDeleted) {
101                    if (nextSessions.length > 0) {
102                        await selectSession(nextSessions[0].id);
103                    } else {
104                        startNewChat();
105                    }
106                }
107            }
108
109            showToast({
110                title: "문서 삭제 완료",
111                description: "문서가 삭제되었습니다.",
112                tone: "success",
113            });
114        } catch (error) {
115            showToast({
116                title: "문서 삭제 실패",
117                description: error instanceof Error ? error.message : "문서 삭제 중 오류가 발생했습니다.",
118                tone: "error",
119            });
120        }
121    };

```

#### ▲ 문서 삭제 및 세션 상태 동기화

해당 코드는 문서 삭제 기능을 처리하는 로직이다. 문서 삭제 시 연결된 채팅 세션이 함께 삭제되었는지 확인하고, 필요할 경우 세션 목록과 현재 선택 상태를 갱신하도록 구현하였다.

DocumentsPage.tsx는 업로드된 문서 목록과 문서 요약 결과를 조회하는 페이지이다. 문서 목록 조회, 검색어 기반 필터링, 문서 상태 표시, 선택 문서의 요약 상세 출력, 요약 다운로드, 문서 삭제 기능 등을 수행한다. 이를 통해 사용자가 업로드한 문서와 요약 결과를 별도의 화면에서 효율적으로 관리할 수 있도록 구현하였다.

## 4.9 ChatLayout.tsx

```
73  const searchedSessions = sessions.filter((session) =>
74    session.title.toLowerCase().includes(chatSearchQuery.trim().toLowerCase())
75  );
76
77  const today = new Date();
78
79  const todaySessions = searchedSessions
80    .filter((session) => isSameDay(new Date(session.updatedAt), today))
81    .slice(0, 5);
82
83  const yesterdaySessions = searchedSessions
84    .filter((session) => getDaysDiff(new Date(session.updatedAt), today) === 1)
85    .slice(0, 5);
86
87  const last7DaysSessions = searchedSessions
88    .filter((session) => {
89      const diff = getDaysDiff(new Date(session.updatedAt), today);
90      return diff >= 2 && diff <= 7;
91    })
92    .slice(0, 5);
```

### ▲ 채팅 세션 검색 및 최근 채팅 분류

해당 코드는 채팅 세션을 검색하고, 오늘·어제·최근 7일 기준으로 최근 대화 목록을 분류하는 로직이다. 이를 통해 사용자가 이전 대화 내역에 보다 쉽게 접근할 수 있도록 구성하였다.

```
229  <div className="grid grid-cols-2 gap-2">
230    <Button
231      variant={panel === "chat" ? "default" : "outline"}
232      size="sm"
233      onClick={() => setPanel("chat")}
234    >
235      채팅
236    </Button>
237    <Button
238      variant={panel === "documents" ? "default" : "outline"}
239      size="sm"
240      onClick={() => setPanel("documents")}
241    >
242      <FileText className="mr-1 size-3" />
243      문서함
244    </Button>
245  </div> /.grid.grid-cols-2.gap-2
246
247  <button
248    type="button"
249    onClick={() => setSearchOverlayOpen(true)}
250    className="relative mt-3 flex h-9 w-full items-center rounded-lg border border-input
251  >
252    <Search className="mr-2 size-4 shrink-0" />
253    <span className="truncate">채팅 검색</span>
254  </button> /.relative.mt-3.flex.h-9.w-full.items-center.rounded-lg.border.border-input.bg-#f3f4f6
```

### ▲ 사이드바 패널 전환 및 검색 진입

해당 코드는 좌측 사이드바에서 채팅, 문서함 패널을 전환하고 검색 오버레이를 여는 로직이다. 사용자가 주요 기능 화면으로 쉽게 이동할 수 있도록 공통 탐색 구조를 제공하도록 하였다.



ChatLayout.tsx는 일반 사용자 화면의 공통 레이아웃을 담당하는 컴포넌트이다. 채팅 세션 목록, 최근 채팅 팝오버, 검색 오버레이, 사용자 정보 표시 기능 등을 포함하며, 사이드바 확장 및 축소 기능을 통해 화면 활용성을 높이도록 설계하였다. 공통 탐색 구조를 레이아웃 단위로 분리함으로써 채팅 화면과 문서 화면이 동일한 사용자 경험을 유지할 수 있도록 하였다.

#### 4.10 uiStore.ts

```
24 export const useUiStore = create<UiState>()(
25   persist(
26     (set) => ({
27       panel: "chat",
28       chatSearchQuery: "",
29       documentSearchQuery: "",
30       sidebarMode: "expanded",
31       recentChatsOpen: false,
32       searchOverlayOpen: false,
33
34       setPanel: (panel) => set({ panel }),
35       setChatSearchQuery: (value) => set({ chatSearchQuery: value }),
36       setDocumentSearchQuery: (value) => set({ documentSearchQuery: value }),
37       setSidebarMode: (mode) => set({ sidebarMode: mode }),
38       toggleSidebar: () =>
39         set((state) => ({
40           sidebarMode: state.sidebarMode === "expanded" ? "collapsed" : "expanded",
41         })),
42       setRecentChatsOpen: (open) => set({ recentChatsOpen: open }),
43       setSearchOverlayOpen: (open) => set({ searchOverlayOpen: open }),
44
45       reset: () =>
46         set({
47           panel: "chat",
48           chatSearchQuery: "",
49           documentSearchQuery: "",
50           sidebarMode: "expanded",
51           recentChatsOpen: false,
52           searchOverlayOpen: false,
53         }),
54     }),
55     {
56       name: "ui-store",
57       partialize: (state) => ({
58         sidebarMode: state.sidebarMode,
59       }),
60     }
61   )
62 )
```

##### ▲ UI 전역 상태 관리

해당 코드는 패널 전환, 검색어 상태, 사이드바 확장 및 축소 상태, 최근 채팅 창 열림 여부 등 UI 관련 전역 상태를 관리하는 로직이다. 일부 상태는 로컬 저장소에 유지되도록 구성하여 사용자 편의성을 높였다.

uiStore.ts는 화면 상태와 UI 인터랙션 상태를 전역으로 관리하는 스토어이다. 현재 패널 상태, 채팅 검색어, 문서 검색어, 사이드바 상태, 최근 채팅 팝오버 열림 여부, 검색 오버레이 열림 여부 등을 관리한다. 이를 통해 UI 관련 상태를 개별 컴포넌트 내부에 분산시키지 않고 공통 상태로 관리할 수 있도록 구성하였다.

## 5. 구현 결과 및 특징

본 프론트엔드 구현의 주요 특징은 다음과 같다.

- React, TypeScript, Zustand를 활용하여 기능별 책임이 명확한 구조로 설계하였다.
- 인증 상태와 채팅 상태를 전역에서 통합 관리하여 화면 간 일관성을 확보하였다.
- WebSocket 기반 채팅 흐름을 적용하여 실시간 응답 처리가 가능하도록 구현하였다.
- 문서 업로드와 요약 결과 확인 기능을 채팅 흐름과 연계하여 사용자 편의성을 향상시켰다.
- 자동 스크롤, 타이핑 애니메이션, 검색 오버레이, 사이드바 제어 등 사용자 경험을 고려한 UI 기능을 반영하였다.

## 6. 결론

프론트엔드 영역에서는 사용자 인증, 채팅 세션 관리, 문서 업로드 및 요약 결과 표시, 실시간 웹소켓 통신 기능을 중심으로 구현을 진행하였다.

또한 상태관리 계층과 화면 계층을 분리하여 유지보수성과 확장성을 높였으며, 사용자 관점에서 직관적으로 사용할 수 있는 인터페이스를 제공하고자 하였다. 이를 통해 기능 구현뿐 아니라 실제 사용성을 고려한 프론트엔드 구조를 설계할 수 있었다.